



Escuela Técnica Superior de
Ingeniería Informática

SEGURIDAD EN SISTEMAS INFORMÁTICOS Y EN INTERNET

**PAI-4. AUDITVUL. ANÁLISIS DE VULNERABILIDADES Y
DETECCIÓN DE INTRUSOS PARA SISTEMAS INFORMÁTICOS**

SECURITY TEAM 2

Alejandro Fernández Orozco
Alejandro Morilla García

Introducción	3
Descripción de OWASP Multidae II	3
Tarea 1: Configurar OpenVAS/Greenbone	4
Preparación del Entorno	4
Descarga e Instalación	4
Verificación y Acceso	4
Actualización de Feeds de Vulnerabilidades	5
Alta de Activos y Configuración de Escaneos	5
Registro de Targets	5
Definición de Escaneos	5
Revisión de Resultados	6
Tarea 2: Análisis de Vulnerabilidades de servidor propio	7
Tarea 3: Definición de un plan de mitigación	10
Tarea 4: Instalación y Configuración de Suricata	12
Tarea 5: Configuración de reglas y lanzamiento de Suricata	12
Tarea 6: Monitorización de Suricata	13
Anexo	14
Fig. 1	14
Fig. 2	14
Fig. 3	15
Fig. 4	15
Fig. 5	16
Fig. 6	16
Fig. 7	16
Fig. 8	16
Fig. 9	17

Introducción

Dentro del Plan Director para la Seguridad de la Información de las empresas y organizaciones se han desarrollado ya varios Proyectos de Seguridad de la información que abordaban el problema de la seguridad de las transmisiones de la información. Además, ya hemos tratado en proyectos anteriores la problemática de que las empresas y organizaciones utilicen equipos clientes (de escritorio y móviles) propios y cómo asegurar servicios que se exponen externamente. Sin embargo, tanto los clientes como las organizaciones no son conscientes, a veces, de los problemas relacionados con la seguridad que existen ya que por un fallo de seguridad tanto en los sistemas de información que usan los clientes/servidores o por no validar/tratar adecuadamente las entradas/salidas de dichas aplicaciones que exponen los servicios de la organización, se puede producir una pérdida de datos por la empresa u organización con el consecuente perjuicio, sino también una sanción administrativa para la misma.

Es por tanto fundamental llevar a cabo un análisis que permitan AUDITAR los sistemas informáticos que se usan, identificar las posibles VULNERABILIDADES, además de desplegar mecanismos que permitan la DETECCIÓN temprana de intrusiones ante posibles ataques sobre los sistemas de informáticos desplegados por las organizaciones.

Descripción de OWASP Multidae II

El sistema elegido como servidor de pruebas es OWASP Multidae II.

OWASP Mutillidae II es una aplicación web intencionalmente vulnerable desarrollada para prácticas de seguridad informática. Creada por la comunidad OWASP, permite a los usuarios aprender y probar técnicas de evaluación de seguridad, como pruebas de penetración y análisis de vulnerabilidades. Mutillidae II simula un entorno web típico con una amplia gama de vulnerabilidades, como inyección SQL, XSS (Cross-Site Scripting), inclusión de archivos y fallos de configuración de seguridad. Además, incluye herramientas educativas como guías integradas y sugerencias para mitigar las vulnerabilidades identificadas. Esta aplicación es ampliamente utilizada para experimentos en entornos controlados y con fines educativos.

Tarea 1: Configurar OpenVAS/Greenbone

En esta tarea se implementará OpenVAS/Greenbone en un entorno Docker utilizando una configuración no persistente. OpenVAS es una herramienta robusta para la detección de vulnerabilidades, que permite analizar sistemas, identificar riesgos de seguridad y proporcionar recomendaciones de mitigación. A continuación, se describen los pasos realizados:

Preparación del Entorno

Para esta instalación se utilizó Docker, una plataforma que facilita la creación de contenedores aislados. Esto permite ejecutar OpenVAS en un entorno controlado e independiente. En este caso, se optó por una configuración no persistente, ideal para pruebas temporales o demostraciones, ya que los datos no se conservan al detener el contenedor.

Descarga e Instalación

La imagen oficial de OpenVAS se obtuvo desde Docker Hub mediante el comando:

1. `docker pull immauss/openvas`

Este proceso garantiza la descarga de la versión más reciente y estable. Posteriormente, se ejecutó el contenedor con los siguientes parámetros:

- **Puerto 9392:** Para acceso a la interfaz web.
- **Contraseña de administrador:** `ssii`.
- **Nombre del contenedor:** `openvas`.

El comando utilizado fue:

2. `docker run --detach --publish 9392:9392 -e PASSWORD="ssii" --name openvas immauss/openvas`

Este comando permitió que OpenVAS se iniciara en segundo plano con la configuración especificada.

Verificación y Acceso

Para confirmar que el contenedor estaba activo, se utilizó el comando:

3. `docker ps`

Esto permitió verificar que el servicio **openvas** estaba ejecutándose correctamente. Una vez operativo, se accedió a OpenVAS desde un navegador web en la dirección <http://127.0.0.1:9392>, utilizando las credenciales:

- **Usuario:** **admin**.
- **Contraseña:** **ssii**.

Desde la interfaz, se pudo acceder al panel de control para configurar tareas y gestionar objetivos.

Actualización de Feeds de Vulnerabilidades

Para garantizar la detección de las vulnerabilidades más recientes, se sincronizó la base de datos de pruebas mediante el comando:

4. `docker exec -it openvas /scripts/sync.sh`

Este proceso aseguró que OpenVAS estuviera actualizado con las amenazas conocidas más recientes. ([Fig 1](#))

Alta de Activos y Configuración de Escaneos

Registro de Targets

Para realizar un análisis, se comenzó registrando los sistemas objetivo en **Configuration > Targets**. Este proceso incluyó:

- **Nombre del Target:** Una descripción clara del sistema (por ejemplo, "Servidor Web Mutillidae").
- **IP o Dominio:** Dirección del sistema a analizar.
- **Lista de Puertos:** Se seleccionó **All TCP and Nmap Top 1000 UDP** para un análisis exhaustivo.

Tras completar el formulario, se guardó el Target, quedando listo para su uso en tareas de escaneo. ([Fig. 2](#))

Definición de Escaneos

Desde la sección **Scans > Tasks**, se configuraron tareas de escaneo siguiendo estos pasos:

1. **Creación de la Tarea** ([Fig. 3](#)):
 - Nombre descriptivo, como "Escaneo Completo Mutillidae".
 - Asociación con el Target registrado previamente.
 - Configuración de escaneo: Se seleccionó **Full and Fast**, un perfil que equilibra rapidez y cobertura.

- Escáner por defecto de OpenVAS.
2. **Ejecución:**
- La tarea se guardó y ejecutó desde el panel, iniciando el análisis del sistema objetivo. Durante el proceso, el progreso se monitorea desde la interfaz [\(Fig. 4\)](#)

Revisión de Resultados

Finalizado el escaneo, OpenVAS generó un informe con:

- **Vulnerabilidades Detectadas:** Clasificadas por severidad (Alta, Media, Baja).
- **Códigos CVE Asociados:** Identificadores únicos para profundizar en los detalles de las vulnerabilidades.
- **Recomendaciones de Mitigación:** Sugerencias prácticas para corregir los problemas detectados.

El informe se consultó desde **Scans > Reports**, donde se analizaron y priorizaron las vulnerabilidades según su criticidad e impacto en el sistema.

Tarea 2: Análisis de Vulnerabilidades de servidor propio

Vulnerabilidad	Servidor Web	Puerto alojado	Sistema Operativo
Redis Server No Password	OWASP Multidae II	6379	Windows 11 pro
MQTT Broker Does Not Require Authentication	OWASP Multidae II	1883	Windows 11 pro
SSL/TLS: Unknown Untrusted / Dangerous Certificate Authority (CA) Detection	OWASP Multidae II	25	Windows 11 pro
SSL/TLS: Deprecated TLSv1.0 and TLSv1.1 Protocol Detection	OWASP Multidae II	9390	Windows 11 pro

Redis Server No Password (CVE-2020-4670)

El servidor Redis, un almacén de estructura de datos, ejecutado en el host remoto no está protegido mediante autenticación con contraseña. Un atacante podría explotar esta vulnerabilidad para obtener acceso no autorizado al servidor.

Se puede utilizar el siguiente exploit: <https://www.exploit-db.com/exploits/47195>

Según NIST: NVD podemos caracterizar esta vulnerabilidad como crítica con una puntuación de 9.1.

Vector: CVSS:3.1 / AV:N / AC:L / PR:N / UI:N / S:U / C:H / I:H / A:N

Según CNA: IMB podemos caracterizar esta vulnerabilidad como alta con una puntuación de 7.4.

Vector: CVSS:3.0 / AV:N / AC:H / PR:N / UI:N / S:U / C:H / I:H / A:N

EXPLOITS DB para buscar los exploits concretos de un cve

MQTT Broker Does Not Require Authentication (CVE-2023-3028)

La autenticación insuficiente en el backend de MQTT permite al atacante acceder y manipular datos de telemetría de una flota completa de vehículos que usan la unidad telemática HopeChart HQT-401. Otros modelos posiblemente también estén afectados. Se han identificado múltiples vulnerabilidades:

- El backend de MQTT no requiere autenticación permitiendo así conexiones no autorizadas por parte de un atacante.

- Los vehículos publican sus datos de telemetría (por ejemplo, ubicación GPS, velocidad, odómetro, combustible, etc.) como mensajes en temas públicos. El backend también envía comandos a los vehículos a medida que MQTT publica en temas públicos. Como resultado, un atacante puede acceder a los datos confidenciales de toda la flota administrada por el backend.

- Los mensajes MQTT enviados por los vehículos o el backend no están cifrados ni autenticados. Un atacante puede crear y publicar mensajes para hacerse pasar por un vehículo o el backend. El atacante podría entonces, por ejemplo, enviar información incorrecta al servidor sobre la ubicación del vehículo.

- El backend puede inyectar datos en el bus CAN de un vehículo enviando un mensaje MQTT específico sobre un tema público. Debido a que estos mensajes no están autenticados ni cifrados, un atacante podría hacerse pasar por el backend, crear un mensaje falso e inyectar datos CAN en cualquier vehículo administrado por el backend.

Se puede utilizar el siguiente exploit: <https://www.exploit-db.com/exploits/50713>

Según NIST: NVD podemos caracterizar esta vulnerabilidad como crítica obteniendo una puntuación de 9.8.

Vector: CVSS:3.1 / AV:N / AC:L / PR:N / UI:N / S:U / C:H / I:H / A:H

Según CNA: Automotive Security Research Group podemos caracterizar esta vulnerabilidad como alta obteniendo una puntuación de 8.6.

Vector: CVSS:3.1 / AV:N / AC:L / PR:N / UI:N / S:U / C:L / I:L / A:H

SSL/TLS: Unknown Untrusted / Dangerous Certificate Authority (CA) Detection (CVE-2022-21672)

Make-ca es una utilidad para proporcionar y gestionar una configuración completa de PKI para estaciones de trabajo y servidores. A partir de la versión 0.9 y antes de la versión 1.10, make-ca interpreta incorrectamente el archivo certdata.txt de Mozilla y trata los certificados explícitamente no confiables como si fueran confiables, lo que provoca que dichos certificados no confiables sean reconocidos como confiables por el sistema. Algunos de estos certificados no confiables eran utilizados por autoridades de certificación (CAs) que ya

habían sido comprometidas. Los atacantes malintencionados podrían realizar un ataque de tipo "Man-in-the-Middle" (MIM) explotando esta vulnerabilidad.

Según CNA: GitHub, Inc. podemos caracterizar esta vulnerabilidad como media, obteniendo una puntuación de 6.5.

Vector: CVSS:3.1 / AV:N / AC:L / PR:L / UI:N / S:U / C:N / I:H / A:N

SSL/TLS: Deprecated TLSv1.0 and TLSv1.1 Protocol Detection (CVE-2011-3389 y CVE-2015-0204)

Esta vulnerabilidad contiene a su vez estas dos vulnerabilidades:

-CVE-2011-3389: Browser Exploit Against SSL/TLS (BEAST): El protocolo SSL, como se utiliza en ciertas configuraciones en Microsoft Windows y Microsoft Internet Explorer, Mozilla Firefox, Google Chrome, Opera y otros productos, cifra los datos mediante el uso del modo CBC con vectores de inicialización encadenados, lo que permite a atacantes man-in-the-middle obtener cabeceras HTTP en texto plano a través de un ataque blockwise chosen-boundary (BCBA) en una sesión HTTPS, junto con el código de JavaScript que usa (1) la API WebSocket HTML5, (2) la API Java URLConnection o (3) la API Silverlight WebClient, también conocido como un ataque "BEAST". Según CVSS v2.0 obtenemos una puntuación de 4.3 categorizando como severidad media.

Vector: AV:N / AC:M / Au:N / C:P / I:N / A:N

-CVE-2015-0204: Factoring Attack on RSA-EXPORT Keys Padding Oracle On Downgraded Legacy Encryption (FREAK): La función `ssl3_get_key_exchange` en `s3_clnt.c` en OpenSSL en versiones anteriores a 0.9.8zd, 1.0.0 en versiones anteriores a 1.0.0p y 1.0.1 en versiones anteriores a 1.0.1k permite a servidores SSL remotos llevar a cabo ataques de degradación de versión RSA-a-EXPORT_RSA y facilitar el descifrado de fuerza bruta ofreciendo una clave RSA efímera débil en un rol no sumiso, relacionado con el caso "FREAK". Según CVSS v2.0 obtenemos una puntuación de 4.3 categorizando como severidad media.

Vector: AV:N / AC:M / Au:N / C:N / I:P / A:N

Tarea 3: Definición de un plan de mitigación

Para definir un buen plan de contingencia debemos tener en cuenta tres aspectos fundamentales: ¿Qué podría pasar?, ¿Qué haremos en respuesta? y ¿Cómo podemos prepararnos con anterioridad?

A continuación el plan de contingencia desarrollado para nuestro sistema de información:

1. Análisis de riesgos

- Amenazas identificadas: fallos del servidor, ataques DDoS, ransomware, inyección SQL, corrupción de datos o errores humanos.
- Impacto potencial: pérdida de datos, interrupción del servicio, accesos no autorizados.

2. Objetivos del plan

- Minimizar el tiempo de inactividad.
- Proteger la integridad y disponibilidad de los datos.
- Garantizar una rápida recuperación ante incidentes.

3. Mecanismos de recuperación

3.1. Copias de seguridad

Se realizarán copias completas semanalmente e incrementales diariamente, programadas en horarios de baja carga para minimizar el impacto en el rendimiento. Se aplicará la política 3-2-1, almacenando copias localmente en diferentes medios y en la nube (AWS S3 o Google Cloud). Los respaldos estarán encriptados con AES-256, y solo personal autorizado podrá acceder a ellos. Se realizarán simulaciones regulares para garantizar su funcionalidad.

3.2. Manual de restauración de sistemas

La restauración incluirá:

- **Base de datos:** Importación de copias a un servidor limpio.
- **Aplicación web:** Reinstalación de dependencias y configuración de servicios críticos.
- **Automatización:** Scripts para agilizar tareas recurrentes.

Se atenderán tanto fallos parciales como restauraciones totales, siguiendo un checklist que documente las dependencias clave y roles asignados.

3.3. Configuración de logs

Se registrarán accesos, errores y actividades críticas con herramientas como Monolog (PHP), Syslog, o Splunk. Los registros se conservarán por al menos seis meses, configurando alertas en tiempo real para detectar actividades sospechosas, como intentos de login fallidos..

3.4. Análisis de fallos de seguridad

Se realizarán auditorías de código y pruebas de penetración periódicas. Los incidentes se detectarán con IDS/IPS (e.g., Suricata) y se gestionarán aislando sistemas afectados, analizando registros con herramientas forenses como Autopsy. Finalmente, se documentará la causa raíz y se aplicarán medidas preventivas tras reuniones post-mortem.

4. Roles y responsabilidades

El equipo de respuesta incluye:

- **Administrador de sistemas:** Responsable de restaurar servidores y servicios.
- **Analista de seguridad:** Gestiona la contención de incidentes y el análisis forense.
- **Responsable de comunicación:** Informa a usuarios y stakeholders de forma clara y oportuna.

5. Procedimientos de comunicación

Se establecerán notificaciones internas para activar el plan de contingencia y coordinar al equipo técnico. Paralelamente, se enviarán mensajes predefinidos a los usuarios afectados, explicando el incidente y los tiempos estimados de resolución.

6. Pruebas y actualizaciones

Se realizarán simulaciones periódicas de incidentes para validar la efectividad del plan. Este será actualizado tras cada prueba o ante cambios significativos en la infraestructura.

Tarea 4: Instalación y Configuración de Suricata

Hemos elegido el sistema Ubuntu 22.10 ejecutado en una máquina virtual. Para la instalación de Suricata hemos optado por la opción de instalación nativa en el sistema operativo.

Para ello ejecutamos `sudo apt update` y posteriormente `sudo apt install suricata`. Una vez finalizada la ejecución de este último comando nos dirigimos al directorio `/etc/suricata` y en la sección `'vars'` y `'address-groups'` modificamos el parámetro `'HOME_NET'` por nuestra red que en este caso es `10.0.2.0/24` (Fig. 5)

Tarea 5: Configuración de reglas y lanzamiento de Suricata

Para crear un conjunto de reglas debemos crear primero el archivo en el que estarán todo el conjunto de ellas. Creamos este archivo y dentro de él escribimos las reglas necesarias para cumplir con la siguiente Política de Seguridad: detectar posible tráfico sospechoso en sus servicios internos desplegados en `http/https` por los puertos `8083`, `8443`, `mysql` por el puerto de administración `3336`, `ssh/sftp` por el puerto `2288`. Todo el tráfico dirigido a estos servicios que no provenga de una red conocida debe ser detectado y marcado como sospechoso. (Fig. 6)

Hemos escrito una regla por cada puerto ya que cada uno se refiere a un protocolo distinto y esto ha de indicarse en el apartado `"msg"`.

Todas las reglas siguen el siguiente patrón:

- `alert`: indica que la acción a realizar, en este caso la regla alertará cuando se cumpla lo siguiente.
- `tcp`: indica el tipo del protocolo del paquete
- `$EXTERNAL_NET any`: estas dos opciones indican que el origen del paquete sea de cualquier dirección ip externa a la red mediante cualquier puerto
- `$HOME_NET`: indica que el destino del paquete es la variable que configuramos anteriormente y es nuestra red.
- `8083,8443, 3336 y 2288`: indican que el puerto destino es el indicado en cada regla.
- `(msg: "")`: indica el mensaje que debe aparecer cuando se alerte.

Al ejecutar el comando para comprobar que no hay errores podemos observar que se han cargado todas las reglas. (Fig. 7)

Tarea 6: Monitorización de Suricata

Para poder probar y corroborar el correcto funcionamiento de todas las reglas nos hemos conectado desde otra máquina y podemos observar como se registra en el archivo fast.log (Fig 8)

Para realizar el informe mensual hemos simulado el tráfico de todo un mes (Fig. 9) y obtenemos la siguiente información:

Informe Mensual de Actividades del IDS (Intrusion Detection System)

Resumen de Alertas Detectadas: Noviembre de 2024

1. Tráfico Sospechoso en Puertos Específicos:

- Total de Alertas: 42

1.1. Tráfico Sospechoso en Puertos HTTP:

- Número de Alertas: 7
- IPs de origen de las alertas: 10.0.2.17

1.2. Tráfico Sospechoso en Puertos HTTPS:

- Número de Alertas: 15
- IPs de origen de las alertas: 10.0.2.17

1.3. Tráfico Sospechoso en el Puerto MySQL:

- Número de Alertas: 12
- IPs de origen de las alertas: 10.0.2.17

1.4. Tráfico Sospechoso en Puertos SSH/SFTP:

- Número de Alertas: 8
- IPs de origen de las alertas: 10.0.2.1

Anexo

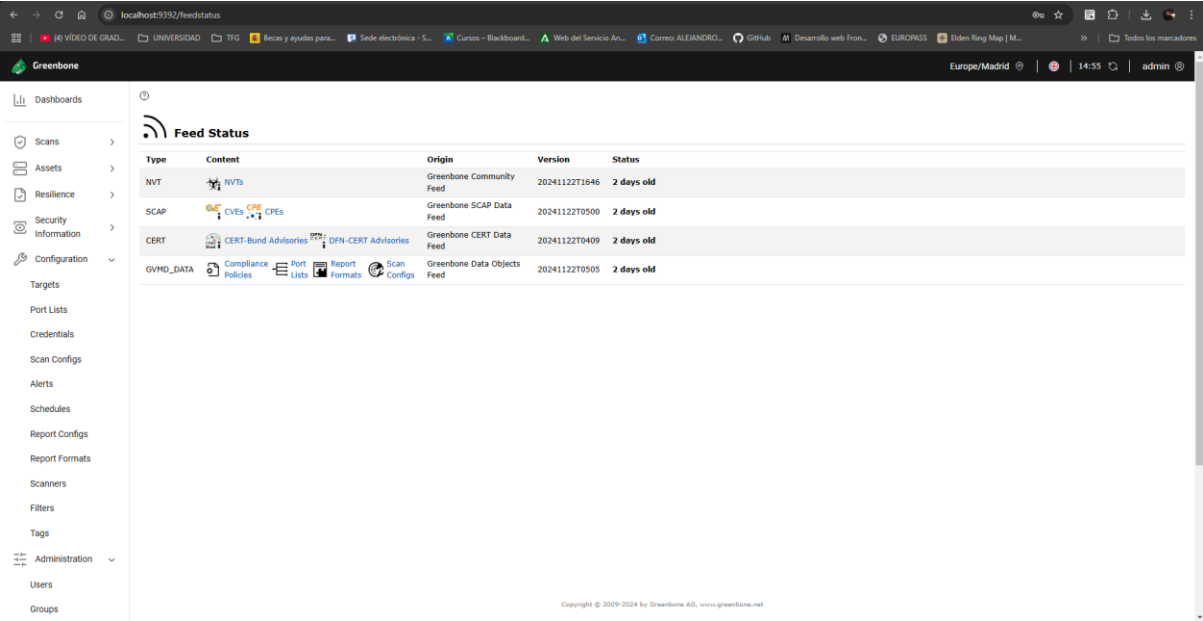


Fig. 1

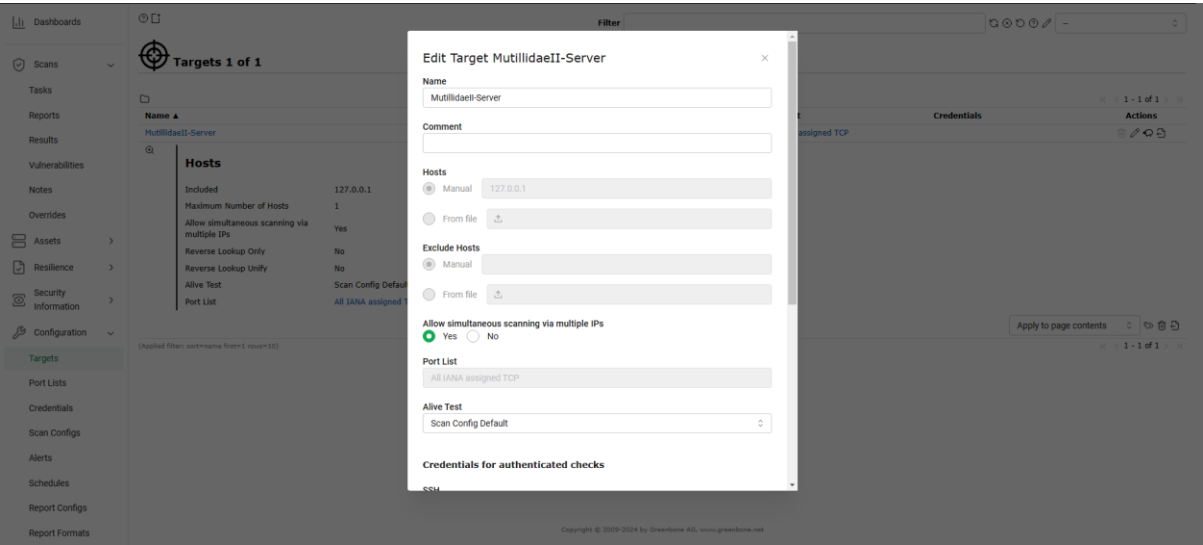


Fig. 2


```

GNU nano 8.1 suricata.yaml *
%YAML 1.1
---

# Suricata configuration file. In addition to the comments describing all
# options in this file, full documentation can be found at:
# https://docs.suricata.io/en/latest/configuration/suricata-yaml.html

# This configuration file generated by Suricata 7.0.6.
suricata-version: "7.0"

##
## Step 1: Inform Suricata about your network
##

vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    #HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
    #HOME_NET: "[192.168.0.0/16]"
    HOME_NET: "[10.0.2.0/24]"
    #HOME_NET: "[172.16.0.0/12]"
    #HOME_NET: "any"

    EXTERNAL_NET: "!$HOME_NET"
    #EXTERNAL_NET: "any"

    HTTP_SERVERS: "$HOME_NET"
    SMTP_SERVERS: "$HOME_NET"
    SQL_SERVERS: "$HOME_NET"
    DNS_SERVERS: "$HOME_NET"
    TELNET_SERVERS: "$HOME_NET"
    AIM_SERVERS: "$EXTERNAL_NET"
    DC_SERVERS: "$HOME_NET"

^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^G Ubicación  ^U Deshacer   ^M Poner marca
^X Salir      ^R Leer fich. ^N Reemplazar ^U Pegar      ^J Justificar ^I Ir a línea ^E Rehacer    ^L Copiar

```

Fig. 5

```

GNU nano 8.1 tarea5.rules *
#Regla para HTTP en puerto 8083
alert tcp $EXTERNAL_NET any -> $HOME_NET 8083 (msg:"Posible tráfico sospechoso HTTP no autorizado"; sid:1000001; priority:1;)
#Regla para HTTPS en puerto 8083
alert tcp $EXTERNAL_NET any -> $HOME_NET 8443 (msg:"Posible tráfico sospechoso HTTPS no autorizado"; sid:1000001; priority:1;)
#Regla para MySQL en puerto 3336
alert tcp $EXTERNAL_NET any -> $HOME_NET 3336 (msg:"Posible tráfico sospechoso MySQL no autorizado"; sid:1000002; priority:1;)
#Regla para SSH/SFTP en puerto 2288
alert tcp $EXTERNAL_NET any -> $HOME_NET 2288 (msg:"Posible tráfico sospechoso SSH/SFTP no autorizado"; sid:1000003; priority:1;)

```

Fig. 6

```

Notice: suricata: This is Suricata version 7.0.2 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 4
Info: Suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: ioctl: enp8s3: MTU 1500
Info: conf: Running in live mode, activating unix socket
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 1 rule files processed, 35323 rules successfully loaded, 0 rules failed
Info: threshold-conf: Threshold config parsed: 0 rule(s) found
Info: detect: 35326 signatures processed. 1269 are IP-only rules, 5291 are inspecting packet payload, 28553 inspect application layer, 108 are decoder event only

```

Fig. 7

```

[**] [1:2108498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] (TCP) 18.154.22.96:80 -> 10.0.2.15:55288
[**] [1:2108498:7] GPL ATTACK_RESPONSE id check returned root [**] [Classification: Potentially Bad Traffic] [Priority: 2] (TCP) 18.154.22.70:80 -> 10.0.2.15:54262
[**] [1:4242:1] Tráfico sospechoso en puertos HTTP [**] [Classification: Potentially Bad Traffic] [Priority: 2] (TCP) 10.0.2.17:1234 -> 10.0.2.15:8083
[**] [1:2324:1] Tráfico sospechoso en puertos HTTPS [**] [Classification: Potentially Bad Traffic] [Priority: 2] (TCP) 10.0.2.17:1234 -> 10.0.2.15:8443
[**] [1:1002:1] Tráfico sospechoso en el puerto MySQL [**] [Classification: Potentially Bad Traffic] [Priority: 2] (TCP) 10.0.2.17:1234 -> 10.0.2.15:3336
[**] [1:1003:1] Tráfico sospechoso en puertos SSH/SFTP [**] [Classification: Potentially Bad Traffic] [Priority: 2] (TCP) 18.154.22.96:1234 -> 10.0.2.15:2288

```

Fig. 8

